



ARQUITETURA DE SOFTWARE

Apostila da disciplina (BES, ADS)

Autor: Prof. Me. Marlon A. Peron Generoso

Universidade Positivo

CAPÍTULO 1: MODELAGEM DE REQUISITOS FUNCIONAIS

Introdução

Antes de projetar a arquitetura de um sistema é necessário compreender claramente quais problemas o software deverá resolver. A Engenharia de Requisitos é a área responsável por identificar, documentar, analisar, validar e gerenciar essas necessidades. Ela funciona como uma ponte entre o problema de negócio e a solução tecnológica.

Quando requisitos são mal compreendidos, erros propagam-se para todas as etapas seguintes do desenvolvimento. Uma decisão arquitetural baseada em requisitos incorretos pode gerar retrabalho, aumento de custos e sistemas que não atendem aos usuários.

O Papel dos Requisitos na Arquitetura de Software

A arquitetura de software descreve a organização de componentes, módulos, interfaces e decisões estruturais de um sistema. Entretanto, nenhuma decisão arquitetural deve ser tomada sem compreender as necessidades do negócio.

Os requisitos funcionam como direcionadores da arquitetura. Eles ajudam a definir quais módulos deverão existir, quais dados deverão ser armazenados, quais integrações serão necessárias e quais mecanismos de comunicação deverão ser implementados.

Por exemplo, se um sistema precisa suportar rastreamento em tempo real de entregas, a arquitetura provavelmente necessitará de mecanismos para atualização frequente de localização, processamento de eventos e comunicação entre serviços.

Requisitos Funcionais

Requisitos funcionais descrevem os comportamentos e serviços que o sistema deve fornecer. Eles respondem à pergunta: o que o sistema deve fazer?

Em um sistema de entregas, exemplos incluem cadastrar clientes, registrar pedidos, calcular fretes, processar pagamentos, emitir comprovantes e rastrear entregas.

Um requisito funcional bem escrito deve ser claro, objetivo, verificável e livre de ambiguidades. Quanto mais preciso for o requisito, mais fácil será validar posteriormente se ele foi atendido.

Casos de Uso

Casos de uso representam uma das técnicas mais tradicionais para modelagem de requisitos funcionais. Eles descrevem a interação entre atores e o sistema com o objetivo de alcançar um resultado de valor.

Um ator representa um papel desempenhado por uma pessoa ou sistema externo. Um mesmo indivíduo pode exercer diferentes papéis em contextos distintos.

Os casos de uso permitem compreender não apenas o objetivo final do usuário, mas também a sequência de ações necessárias para alcançá-lo. Por esse motivo, são amplamente utilizados em projetos que exigem maior documentação e rastreabilidade.

Elementos de um Caso de Uso

- Ator: entidade que interage com o sistema.
- Caso de Uso: funcionalidade oferecida pelo sistema.
- Fluxo Principal: caminho de execução esperado.
- Fluxos Alternativos: situações excepcionais ou desvios.

Exemplo de Caso de Uso

- Nome: Realizar Pedido
- Ator: Cliente
- Fluxo Principal:
 - O cliente seleciona os produtos.
 - O sistema calcula o valor total.
 - O cliente informa o endereço de entrega.
 - O sistema calcula o frete.
 - O cliente confirma o pedido.
 - O sistema registra a compra.
- Fluxos Alternativos:
 - Produto indisponível.
 - Endereço inválido.
 - Pagamento recusado.

Histórias de Usuário

As histórias de usuário surgiram no contexto das metodologias ágeis e fornecem uma maneira simples de capturar necessidades sob a perspectiva de quem utilizará o sistema.

Sua principal característica é manter o foco no valor entregue ao usuário. Diferentemente dos casos de uso, elas não procuram descrever detalhadamente cada passo da interação.

Estrutura clássica:

Como [tipo de usuário], eu quero [ação] para [benefício].

Exemplo:

Como **cliente**, eu quero **rastrear minha entrega** para **saber onde meu pedido está**.

Critérios de Aceitação

Uma história de usuário sozinha nem sempre é suficiente para orientar a implementação. Por isso, normalmente são definidos critérios de aceitação.

Critérios de aceitação descrevem condições objetivas que devem ser satisfeitas para que a funcionalidade seja considerada concluída.

Exemplo:

- O sistema deve exibir a localização atual da entrega.
- O sistema deve exibir a previsão de chegada.
- O sistema deve informar alterações de status.
- O sistema deve responder à consulta em até três segundos.

Casos de Uso e Histórias de Usuário

Casos de uso e histórias de usuário não são concorrentes. Na prática, muitas organizações utilizam ambas as abordagens.

Histórias de usuário são eficientes para comunicação com clientes e priorização do backlog.

Casos de uso são úteis quando existe necessidade de maior detalhamento, documentação formal ou análise aprofundada.

Estudo de Caso

Ao longo desta apostila será utilizado um Sistema de Entregas Inteligentes.

História de usuário inicial:

Como cliente, eu quero acompanhar minha entrega para saber quando ela chegará.

A partir dessa necessidade podem ser identificados diversos requisitos funcionais:

- Cadastrar cliente.
- Registrar pedido.
- Calcular frete.
- Registrar pagamento.
- Atualizar status da entrega.
- Consultar rastreamento.
- Emitir comprovante.

Esses requisitos serão utilizados nos próximos capítulos para construção de diagramas UML, documentos arquiteturais e modelos arquitetônicos.

Resumo

Os requisitos representam a base de todo projeto de software. Casos de uso fornecem uma visão detalhada das interações entre usuários e sistema. Histórias de usuário simplificam a comunicação e enfatizam valor de negócio. Ambas as abordagens contribuem para o desenvolvimento de arquiteturas mais alinhadas às necessidades reais dos usuários.

Exercícios

1. Liste cinco requisitos funcionais para um sistema de biblioteca.
2. Crie três histórias de usuário para um sistema de comércio eletrônico.
3. Defina critérios de aceitação para uma funcionalidade de cancelamento de pedidos.
4. Elabore um caso de uso para matrícula em disciplinas.
5. Proponha três novos requisitos para o Sistema de Entregas Inteligentes e explique seus impactos arquiteturais.

CAPÍTULO 2: MODELAGEM DE CLASSES E VISÃO LÓGICA DA ARQUITETURA

Introdução

Após identificar os requisitos funcionais do sistema, o próximo passo consiste em transformar essas necessidades em elementos que possam ser implementados pelos desenvolvedores. Uma das principais ferramentas para realizar essa transição é o modelo de classes, responsável por representar a estrutura estática do sistema.

O modelo de classes permite identificar quais entidades existem no domínio do problema, quais informações precisam ser armazenadas e quais comportamentos devem ser executados por cada elemento do sistema. Ele serve como base para a implementação, para a documentação da arquitetura e para a comunicação entre analistas, arquitetos e desenvolvedores.

Neste capítulo será utilizado o mesmo estudo de caso iniciado no capítulo anterior: o Sistema de Entregas Inteligentes.

A Visão Lógica da Arquitetura

Uma arquitetura de software pode ser observada sob diferentes perspectivas. No modelo arquitetural 4+1 proposto por Philippe Kruchten, a Visão Lógica é responsável por representar a estrutura funcional do sistema do ponto de vista dos desenvolvedores. Ela descreve classes, objetos e relacionamentos por meio de modelos UML.

O objetivo dessa visão é responder perguntas como:

- Quais entidades existem no sistema?
- Quais informações cada entidade deve armazenar?
- Como essas entidades se relacionam?
- Quais comportamentos devem ser implementados?

Ao responder essas questões, o arquiteto cria uma representação conceitual que servirá de base para o desenvolvimento do software.

O Que é um Modelo de Classes?

O modelo de classes é uma representação estática do sistema que mostra classes, atributos, métodos e relacionamentos. Seu objetivo é organizar os elementos do software de forma coerente com os requisitos identificados anteriormente.

Uma classe representa um conjunto de objetos semelhantes.

Por exemplo:

- Cliente
- Pedido
- Produto
- Pagamento
- Entrega

Cada uma dessas entidades possui características específicas e comportamentos próprios.

No contexto do Sistema de Entregas Inteligentes, a classe Cliente representa os consumidores do serviço, enquanto a classe Pedido representa as compras realizadas pelos clientes.

Elementos de uma Classe

Uma classe normalmente possui três componentes principais.

Nome da Classe

Representa a entidade modelada.

Exemplos:

- Cliente
- Pedido
- Entrega
- Produto

O nome deve ser claro e representar um conceito relevante para o negócio.

Atributos

Atributos representam características ou dados pertencentes à classe.

Exemplo:

- Classe Cliente
 - id
 - nome
 - telefone
 - email
- Classe Produto
 - codigo
 - descricao
 - preco
 - estoque

Os atributos representam o estado de um objeto.

Métodos

Métodos representam os comportamentos ou ações que a classe pode executar.

Exemplo:

- Classe Pedido
 - calcularTotal()
 - adicionarProduto()
 - cancelarPedido()
- Classe Entrega
 - atualizarStatus()
 - informarLocalizacao()

Os métodos representam as operações do sistema.

Visibilidade dos Elementos

A UML utiliza símbolos para indicar a visibilidade dos atributos e métodos.

Símbolo	Visibilidade
+	Público

-	Privado
#	Protegido

Na maioria dos projetos orientados a objetos, os atributos são mantidos privados para preservar o encapsulamento e evitar acessos indevidos.

O encapsulamento é um dos princípios fundamentais da orientação a objetos, pois ajuda a proteger os dados internos dos objetos.

Como Identificar Classes

Uma das principais dificuldades da modelagem é descobrir quais classes devem existir.

Existem diversas técnicas para auxiliar esse processo.

Técnica dos Substantivos

Consiste em analisar descrições do problema e destacar os substantivos encontrados. Esses substantivos tendem a representar possíveis classes.

Exemplo:

"O cliente realiza um pedido utilizando produtos cadastrados no sistema."

Substantivos identificados:

- Cliente
- Pedido
- Produto
- Sistema

Possíveis classes:

- Cliente
- Pedido
- Produto

Nem todos os substantivos necessariamente se transformarão em classes.

Técnica dos Verbos

Os verbos normalmente indicam comportamentos e podem originar métodos.

No exemplo anterior:

- Verbos encontrados:
 - realizar
 - utilizar
 - cadastrar
- Possíveis métodos:
 - realizarPedido()
 - cadastrarProduto()

Essa abordagem auxilia na identificação das responsabilidades das classes.

Agrupamento por Responsabilidade

Outra técnica consiste em agrupar funcionalidades relacionadas.

Exemplo:

Responsabilidades relacionadas ao cliente:

- atualizar cadastro
- visualizar pedidos
- consultar entregas

Essas responsabilidades indicam que a classe Cliente deve existir e concentrar tais comportamentos.

Refinamento e Descarte

Nem todos os elementos identificados inicialmente devem permanecer no modelo.

Durante a modelagem, algumas possíveis classes podem ser descartadas por serem:

- muito genéricas;
- redundantes;
- irrelevantes para a solução.

O refinamento é uma etapa essencial para manter o modelo simples e compreensível.

Identificação de Classes a Partir de Requisitos

Os requisitos apresentados no capítulo anterior servem como ponto de partida para a modelagem.

Considere a seguinte história de usuário:

Como cliente, eu quero fazer um pedido online para receber meus produtos em casa.

A partir dela podemos extrair:

- Substantivos:
 - Cliente
 - Pedido
 - Produto
 - Endereço
- Verbos:
 - fazer pedido
 - receber produto
- Possíveis classes:
 - Cliente
 - Pedido
 - Produto
 - Endereco
- Possíveis métodos:
 - criarPedido()
 - calcularValorTotal()
 - atualizarEntrega()

Esse processo demonstra como os requisitos influenciam diretamente a arquitetura do sistema.

Relacionamentos Entre Classes

Após identificar as classes, é necessário determinar como elas se relacionam.

Associação

Representa uma relação estrutural simples entre duas classes.

Exemplo:

Cliente realiza Pedido.

Cliente ----- Pedido

Agregação

Representa uma relação de posse mais fraca.

Exemplo:

Uma empresa possui entregadores.

Os entregadores continuam existindo mesmo que a empresa deixe de existir.

Composição

Representa uma relação de posse forte.

Exemplo:

Um pedido contém itens de pedido.

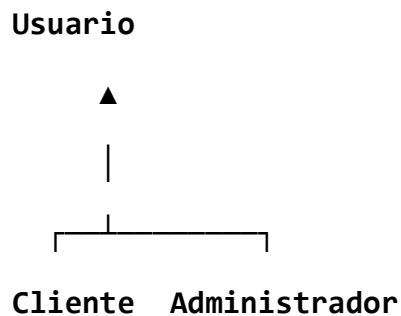
Se o pedido for removido, seus itens também deixam de existir.

Pedido ◆----- **ItemPedido**

Herança

Permite criar especializações de classes mais genéricas.

Exemplo:



A herança favorece reutilização de código e organização do modelo.

Dependência

Representa o uso temporário de uma classe por outra.

Exemplo:

A classe Relatorio utiliza dados da classe Pedido apenas durante sua execução.

Construindo o Modelo de Classes do Sistema de Entregas

A partir dos requisitos levantados, podemos propor o seguinte modelo inicial:

- Classes Principais
 - Cliente
 - id
 - nome
 - email
 - Pedido
 - numero
 - data
 - valorTotal
 - Produto
 - codigo
 - descricao
 - preco
 - Pagamento
 - valor
 - dataPagamento
 - status
 - Entrega
 - codigoRastreamento
 - status
 - localizacaoAtual
- Relacionamentos
 - **Cliente 1 ----- * Pedido**
 - **Pedido 1 ----- * Produto**
 - **Pedido 1 ----- 1 Pagamento**
 - **Pedido 1 ----- 1 Entrega**

Esse modelo servirá de base para os próximos capítulos da apostila.

Boas Práticas de Modelagem

Durante a construção do modelo de classes, recomenda-se:

- Utilizar nomes claros e objetivos.
- Evitar classes excessivamente genéricas.

- Garantir que cada classe tenha uma responsabilidade bem definida.
- Modelar apenas elementos relevantes ao negócio.
- Revisar continuamente o modelo conforme novos requisitos surgirem.
- Manter coerência entre requisitos e modelo estrutural.

Um bom modelo deve ser simples o suficiente para ser compreendido e detalhado o suficiente para orientar o desenvolvimento.

Resumo

Neste capítulo estudamos o modelo de classes, um dos principais mecanismos de representação da Visão Lógica da Arquitetura de Software. Foram apresentados os elementos de uma classe, técnicas para identificação de entidades, tipos de relacionamentos e boas práticas de modelagem. Também foi construída uma primeira versão do modelo de classes do Sistema de Entregas Inteligentes, que servirá de base para os próximos capítulos da disciplina.

Exercícios

1. Descreva cinco possíveis classes para um sistema de biblioteca digital.
2. Utilize a técnica dos substantivos para identificar classes em um sistema de locação de veículos.
3. A partir da história de usuário abaixo, identifique classes e métodos:
Como aluno, eu quero realizar minha matrícula para participar das disciplinas do semestre.
4. Explique a diferença entre associação, agregação e composição.
5. Modele um diagrama de classes para um sistema de clínica médica contendo, no mínimo:
 - Paciente;
 - Médico;
 - Consulta;
 - Prontuário;
 - Receita.
6. No Sistema de Entregas Inteligentes, proponha três novas classes e justifique sua inclusão no modelo arquitetural.

CAPÍTULO 3: DIAGRAMAS DE SEQUÊNCIA

Introdução

O diagrama de sequência é um dos diagramas mais importantes da UML. Ele representa como objetos interagem ao longo do tempo para executar uma funcionalidade específica do sistema. Em outras palavras: ***Diagramas de sequência demonstram como as classes identificadas no diagrama de classes colaboram para resolver um caso de uso.***

Diferentemente do diagrama de classes, que apresenta uma visão estrutural, o diagrama de sequência mostra o comportamento dinâmico do software. É especialmente útil para validar fluxos antes da implementação e para compreender responsabilidades entre objetos.

Importância dos Diagramas de Sequência

A modelagem comportamental permite identificar a ordem exata das operações realizadas por um sistema. Durante a análise e o projeto arquitetural, esses diagramas ajudam a detectar problemas de fluxo, excesso de responsabilidades e dependências inadequadas. Também servem como apoio para desenvolvedores, testadores e arquitetos.

Elementos Principais

- Ator: Entidade externa que interage com o sistema.
- Objetos: instâncias das classes participantes do cenário.
- Linha de Vida: representa a existência dos objetos durante o fluxo.
- Mensagens: chamadas realizadas entre objetos.
- Ativação: período em que um objeto está processando uma operação.
- Criação e Destruição: representam o surgimento ou encerramento de objetos durante a execução.

Tipos de Mensagens

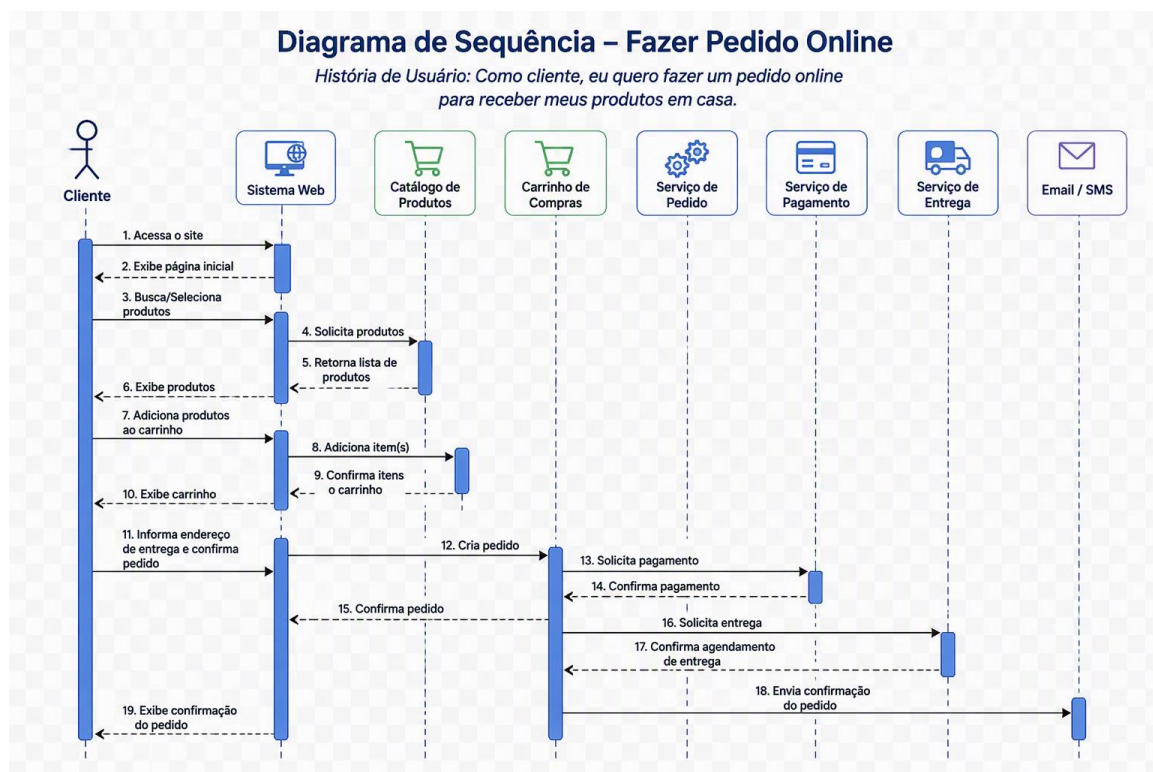
Mensagens síncronas exigem resposta antes da continuidade do fluxo. Mensagens assíncronas permitem que o processamento continue sem aguardar retorno imediato. Mensagens de retorno representam respostas produzidas por uma chamada anterior.

Como Construir um Diagrama de Sequência

Comece escolhendo um caso de uso específico. Em seguida identifique atores e objetos participantes. Defina a ordem temporal das mensagens e registre métodos relevantes. Adicione ativações, retornos e valide a consistência com os requisitos e com o diagrama de classes.

Estudo de Caso: Sistema de Entregas Inteligentes

Considere a funcionalidade de realização de pedidos. O cliente inicia uma solicitação. O controlador recebe a requisição, valida os dados, consulta estoque, registra o pedido e devolve uma confirmação. Esse fluxo torna explícita a cooperação entre componentes do sistema.



Boas Práticas

Utilize nomes claros nas mensagens, modele apenas cenários relevantes, mantenha o fluxo legível da esquerda para a direita e preserve a consistência com os demais artefatos arquiteturais.

Relação com Diagramas de Classes

Os diagramas de sequência complementam os diagramas de classes. Enquanto as classes mostram estrutura, os diagramas de sequência mostram como objetos colaboram em tempo de execução. Ambos devem permanecer alinhados para garantir coerência arquitetural.

Ferramentas de Modelagem

Ferramentas populares incluem Draw.io, Lucidchart, StarUML e PlantUML. Cada uma oferece recursos para representar cenários, mensagens e interações entre objetos.

Resumo

Diagramas de sequência representam o comportamento dinâmico dos sistemas, evidenciando a troca de mensagens entre objetos ao longo do tempo. São fundamentais para validar cenários e apoiar decisões arquiteturais.

Exercícios

1. Construa um diagrama de sequência para Cadastro de Cliente.
2. Crie um diagrama de sequência para rastreamento de entregas.
3. Modele o fluxo de cancelamento de pedido.
4. Represente o processo de autenticação de usuário.
5. Analise como a alteração de requisitos impactaria os diagramas produzidos.

CAPÍTULO 4: VISÃO ARQUITETURAL 4+1

Introdução

Nos capítulos anteriores foram estudados os requisitos funcionais, a modelagem de classes e os diagramas de sequência. Esses artefatos ajudam a compreender o problema e a descrever a solução sob diferentes perspectivas. Entretanto, sistemas reais possuem diversos interessados, como desenvolvedores, arquitetos, gestores, clientes e administradores de infraestrutura. Cada um desses grupos possui preocupações específicas.

Para atender essa necessidade, Philippe Kruchten propôs o modelo arquitetural 4+1, uma das abordagens mais difundidas para documentação e comunicação da arquitetura de software. O modelo organiza a arquitetura em diferentes visões, permitindo que cada stakeholder compreenda os aspectos mais relevantes do sistema.

A Necessidade de Múltiplas Visões

Em projetos complexos, um único diagrama dificilmente consegue representar toda a arquitetura. Enquanto desenvolvedores desejam compreender classes e componentes, equipes de infraestrutura precisam conhecer servidores e implantações. Gestores, por sua vez, procuram entender riscos, organização do projeto e impacto das decisões técnicas.

O modelo 4+1 resolve esse problema organizando a arquitetura em múltiplas perspectivas complementares.

O Modelo 4+1

O modelo é composto por quatro visões principais e uma visão complementar baseada em cenários ou casos de uso.

Visão Lógica

A visão lógica descreve a estrutura funcional do sistema. Seu foco está nas classes, objetos e relacionamentos responsáveis por implementar os requisitos funcionais.

Essa é a visão mais próxima dos modelos UML estudados anteriormente.

No Sistema de Entregas Inteligentes, a visão lógica pode conter classes como Cliente, Pedido, Produto, Entrega e Pagamento.

Questões respondidas pela visão lógica:

- Quais entidades existem?
- Como elas se relacionam?
- Quais responsabilidades pertencem a cada classe?

Visão de Processos

A visão de processos descreve o comportamento dinâmico do sistema durante a execução.

Seu objetivo é representar concorrência, comunicação, sincronização e processamento.

Enquanto o diagrama de classes mostra a estrutura, a visão de processos mostra como os componentes colaboram durante a execução.

No Sistema de Entregas Inteligentes, pode existir um processo responsável pelo gerenciamento de pedidos e outro encarregado das notificações aos clientes.

Questões respondidas:

- Como ocorre a comunicação?
- Quais atividades executam simultaneamente?
- Como a arquitetura lida com desempenho e disponibilidade?

Visão de Desenvolvimento

A visão de desenvolvimento descreve como o software é organizado para construção e manutenção.

Ela apresenta módulos, bibliotecas, componentes, pacotes e estruturas de diretórios utilizadas pelos desenvolvedores.

Um exemplo para o sistema utilizado na apostila poderia ser:

- Modulo de Clientes
- Modulo de Pedidos
- Modulo de Entregas
- Modulo de Pagamentos
- Modulo de Relatórios

Essa visão ajuda a distribuir responsabilidades entre equipes e facilita a evolução do software.

Visão Física

A visão física apresenta a infraestrutura necessária para execução do sistema.

Ela descreve servidores, bancos de dados, redes, dispositivos e elementos de comunicação.

Exemplo simplificado:

Cliente Web -> Servidor de Aplicação -> Banco de Dados

Em arquiteturas modernas podem existir múltiplos servidores, serviços em nuvem, balanceadores de carga e mecanismos de redundância.

Essa visão é especialmente importante para requisitos não funcionais relacionados a desempenho, disponibilidade e escalabilidade.

A Visão de Cenários

A quinta visão é composta pelos cenários e casos de uso mais importantes do sistema.

Ela funciona como elemento integrador das demais visões, permitindo validar se a arquitetura realmente atende às necessidades dos usuários.

Um cenário típico do Sistema de Entregas Inteligentes pode ser:

- O cliente realiza um pedido.
- O sistema registra o pagamento.
- O estoque é atualizado.
- Uma entrega é criada.
- O cliente recebe notificações de rastreamento.

Esse único cenário pode ser observado sob todas as perspectivas do modelo 4+1.

Aplicação ao Estudo de Caso

No contexto da apostila, a evolução do sistema pode ser analisada pelas cinco visões.

1. A visão lógica descreve as entidades do domínio.
2. A visão de processos mostra a comunicação entre componentes.
3. A visão de desenvolvimento organiza módulos e código-fonte.
4. A visão física apresenta a infraestrutura.
5. A visão de cenários valida o comportamento esperado.

Benefícios do Modelo 4+1

- Melhora a comunicação entre stakeholders.
- Reduz ambiguidades na documentação.
- Facilita a manutenção e evolução do software.
- Permite analisar requisitos funcionais e não funcionais.
- Auxilia na construção de Documentos de Arquitetura de Software.

Limitações

- O modelo exige esforço adicional de documentação.
- Projetos muito pequenos podem não necessitar de todas as visões.
- A documentação deve ser mantida atualizada para continuar útil ao longo do ciclo de vida do software.

Preparação para o Próximo Capítulo

Agora que já compreendemos como a arquitetura pode ser observada sob múltiplas perspectivas, estamos preparados para estudar o Documento de Arquitetura de Software (DAS). O DAS utiliza diversas informações produzidas pelas visões arquiteturais para documentar decisões, restrições e justificativas do projeto.

Resumo

O modelo 4+1 é uma abordagem amplamente utilizada para descrição arquitetural. Ele divide a arquitetura em visões lógica, processos, desenvolvimento, física e cenários, permitindo atender diferentes necessidades de informação. Essa abordagem facilita a comunicação, a manutenção e a evolução dos sistemas.

Exercícios

1. Explique as diferenças entre as visões lógica e de processos.
2. Descreva uma possível visão física para um sistema acadêmico online.
3. Identifique os principais módulos de desenvolvimento para um sistema de biblioteca.
4. Escolha um caso de uso estudado anteriormente e explique como ele aparece em cada uma das cinco visões.
5. Proponha uma nova funcionalidade para o Sistema de Entregas Inteligentes e descreva seus impactos em cada visão arquitetural.

CAPÍTULO 5: ELABORAÇÃO DO DOCUMENTO DE ARQUITETURA DE SOFTWARE (DAS)

Introdução

Ao longo dos capítulos anteriores foram produzidos diversos artefatos fundamentais para o desenvolvimento de software. Inicialmente foram identificados os requisitos do sistema, posteriormente foram modelados casos de uso e histórias de usuário, em seguida foi elaborado o modelo de classes, construídos diagramas de sequência e analisada a arquitetura por meio das visões do modelo 4+1. O próximo passo consiste em consolidar essas informações em um documento arquitetural formal.

Esse documento é denominado Documento de Arquitetura de Software (DAS) e representa uma das principais referências técnicas de um projeto de software. Seu objetivo é registrar decisões arquiteturais, justificar escolhas tecnológicas, documentar restrições e servir como fonte de consulta para todos os envolvidos no projeto.

O Que é um DAS?

O Documento de Arquitetura de Software é um artefato utilizado para descrever a arquitetura de um sistema, seus componentes, visões, decisões técnicas e justificativas. Ele funciona como um guia para implementação, manutenção e evolução do software.

Em termos práticos, o DAS responde perguntas como:

1. Qual problema o sistema resolve?
2. Como a solução está organizada?
3. Quais tecnologias serão utilizadas?
4. Quais requisitos arquiteturais devem ser atendidos?
5. Como os componentes interagem?
6. Por que determinadas decisões foram tomadas?

A existência dessas informações reduz ambiguidades e evita a perda de conhecimento ao longo do ciclo de vida do projeto.

Objetivos do DAS

O DAS possui diversos objetivos importantes:

- Comunicação

- O documento permite que desenvolvedores, arquitetos, gerentes, analistas e clientes compartilhem uma mesma visão da solução proposta.
- Padronização
 - Ao documentar decisões e diretrizes, o DAS promove consistência durante a implementação.
- Rastreabilidade
 - Permite relacionar requisitos, decisões arquiteturais e implementações futuras.
- Manutenção
 - Facilita a compreensão do sistema por novos integrantes da equipe e reduz riscos em futuras modificações.
- Governança Arquitetural
 - Auxilia equipes técnicas a verificar se as implementações permanecem alinhadas às decisões arquiteturais originalmente estabelecidas.

Quando Elaborar um DAS?

O DAS normalmente é produzido após a definição da arquitetura e antes do início da implementação principal do sistema.

No entanto, ele não deve ser considerado um documento estático.

Arquiteturas evoluem ao longo do tempo. Novos requisitos surgem, tecnologias são substituídas e decisões são revisadas.

Por esse motivo, o DAS deve ser tratado como um documento vivo.

Relação Entre DAS e o Modelo 4+1

O modelo 4+1 estudado no capítulo anterior é frequentemente utilizado como estrutura organizadora do DAS.

Dentro do documento, normalmente são apresentadas:

- Visão Lógica
- Visão de Processos
- Visão de Desenvolvimento
- Visão Física
- Cenários de Uso

Dessa forma, diferentes stakeholders conseguem localizar rapidamente as informações necessárias para suas atividades.

Padrão IEEE 42010

O IEEE 42010 é um padrão internacional que orienta a descrição de arquiteturas de sistemas e software. Ele estabelece conceitos que ajudam a produzir documentação arquitetural consistente e compreensível.

Segundo esse padrão, uma arquitetura deve considerar:

- Stakeholders: São as partes interessadas no sistema. Exemplos:
 - Clientes
 - Usuários
 - Desenvolvedores
 - Arquitetos
 - Gerentes
 - Equipe de infraestrutura
- Concerns: Representam preocupações dos stakeholders. Exemplos:
 - Segurança
 - Escalabilidade
 - Disponibilidade
 - Usabilidade
 - Desempenho
- Viewpoints: Definem regras para construção das visões arquiteturais.
 - Views: São as representações concretas da arquitetura. Exemplos:
 - Diagramas UML
 - Diagramas de implantação
 - Diagramas de componentes
- Rationales: Correspondem às justificativas das decisões tomadas.

Estrutura Recomendada para um DAS

Embora organizações possam adotar modelos próprios, uma estrutura bastante utilizada é a seguinte:

1. **Introdução:** Apresenta objetivo, escopo e contexto do sistema.
2. **Referências:** Lista normas, documentos e materiais utilizados.
3. **Requisitos Arquiteturais:** Descreve requisitos que impactam a arquitetura.
4. **Visão Lógica:** Contém diagramas de classes e componentes.

5. **Visão de Processos:** Documenta fluxos de execução e concorrência.
6. **Visão de Desenvolvimento:** Mostra módulos, pacotes e organização do código.
7. **Visão Física:** Apresenta infraestrutura e implantação.
8. **Cenários Arquiteturais:** Demonstra funcionamento do sistema em situações importantes.
9. **Decisões Arquiteturais:** Registra escolhas importantes e suas justificativas.
10. **Riscos e Restrições:** Apresenta limitações técnicas e riscos identificados.
11. **Glossário:** Padroniza termos utilizados no projeto.

Registro de Decisões Arquiteturais

Uma das seções mais importantes do DAS é o registro das decisões arquiteturais.

Cada decisão deve explicar:

- O problema identificado.
- As alternativas consideradas.
- A solução escolhida.
- A justificativa da escolha.
- Consequências positivas e negativas.

Exemplo:

Decisão: Utilizar PostgreSQL como banco de dados principal.

Alternativas analisadas:

- MySQL
- PostgreSQL
- SQL Server

Justificativa: Necessidade de confiabilidade, suporte a transações e recursos avançados de consultas.

Impactos: Maior flexibilidade para evolução futura do sistema.

DAS do Sistema de Entregas Inteligentes

Neste momento da apostila já possuímos material suficiente para iniciar a elaboração do DAS do estudo de caso.

Objetivo: Desenvolver uma plataforma de gerenciamento e rastreamento de entregas.

Requisitos Arquiteturais

- Disponibilidade elevada.
- Escalabilidade para milhares de clientes.
- Rastreabilidade de pedidos.
- Segurança dos dados.

Visão Lógica

Classes principais:

- Cliente
- Pedido
- Produto
- Pagamento
- Entrega

Visão de Processos

Fluxo principal:

Cliente → Pedido → Pagamento → Entrega → Notificação.

Visão de Desenvolvimento

Módulos:

- Clientes
- Pedidos
- Pagamentos
- Entregas
- Relatórios

Visão Física

Infraestrutura inicial:

- Aplicação Web
- Servidor de Aplicação
- Banco de Dados
- Serviço de E-mail

Boas Práticas na Elaboração de um DAS

Durante a construção do documento recomenda-se:

- Utilizar linguagem objetiva.
- Registrar decisões importantes.
- Evitar documentação excessiva.
- Manter diagramas atualizados.
- Revisar o documento periodicamente.
- Garantir alinhamento com requisitos.
- Identificar riscos arquiteturais precocemente.

Um DAS eficiente não é necessariamente o mais longo, mas aquele que fornece informações relevantes para apoiar decisões futuras.

Preparação para o Próximo Capítulo

Neste capítulo foi produzido o principal artefato de documentação arquitetural do projeto.

A partir dele, torna-se possível avançar para a implementação da solução.

No próximo capítulo iniciaremos a implementação arquitetural utilizando o padrão MVC, observando como as decisões documentadas no DAS influenciam diretamente a organização do código.

Resumo

O Documento de Arquitetura de Software é o principal artefato de documentação arquitetural de um projeto. Ele consolida requisitos, visões arquiteturais, decisões técnicas e justificativas, funcionando como referência para implementação e manutenção do sistema. Sua estrutura é fortemente influenciada pelo modelo 4+1 e pelas recomendações do IEEE 42010.

Exercícios

1. Elabore uma seção de introdução para o DAS de um sistema acadêmico.
2. Identifique cinco stakeholders e cinco concerns para um sistema bancário.
3. Proponha três decisões arquiteturais para o Sistema de Entregas Inteligentes e registre suas justificativas.

4. Descreva uma visão física para um sistema de comércio eletrônico de grande porte.
5. Produza um mini-DAS contendo Introdução, Requisitos Arquiteturais, Visão Lógica e Visão Física para um sistema de biblioteca digital.

CAPÍTULO 6: IMPLEMENTAÇÃO UTILIZANDO MVC

Introdução

Até este ponto da apostila construímos os principais artefatos de análise e projeto do Sistema de Entregas Inteligentes.

Inicialmente foram levantados os requisitos do sistema. Em seguida, modelamos casos de uso, histórias de usuário, diagramas de classes e diagramas de sequência. Posteriormente estudamos o modelo arquitetural 4+1 e consolidamos as decisões arquiteturais no Documento de Arquitetura de Software (DAS).

Agora chegamos ao momento em que a arquitetura começa a se transformar em software executável.

Para isso, estudaremos um dos padrões arquiteturais mais utilizados no desenvolvimento de aplicações corporativas: o MVC (Model-View-Controller). Esse padrão fornece uma estrutura organizada para separar responsabilidades e reduzir o acoplamento entre componentes da aplicação.

O Que é MVC?

MVC é a sigla para:

- Model
- View
- Controller

O padrão surgiu com o objetivo de separar diferentes responsabilidades da aplicação em componentes independentes, tornando o sistema mais organizado, manutenível e escalável.

Antes do MVC era comum encontrar aplicações contendo regras de negócio, acesso a banco de dados e interface misturados em um único bloco de código.

Isso dificultava:

- manutenção;
- testes;
- reutilização;
- evolução da aplicação.

O MVC propõe uma divisão clara dessas responsabilidades.

Motivação

Imagine um sistema de entregas onde a tela de cadastro de pedidos realiza diretamente:

- a validação dos dados;
- o cálculo do frete;
- o acesso ao banco;
- a geração da resposta.

Nesse cenário a interface passa a conhecer toda a lógica da aplicação.

Qualquer modificação futura exigirá alterações em diversos pontos do sistema.

O MVC resolve esse problema distribuindo responsabilidades.

Componente Model

O Model representa os dados e as regras de negócio da aplicação.

É nele que encontramos:

- entidades;
- validações;
- cálculos;
- comportamentos de negócio.

No Sistema de Entregas Inteligentes algumas entidades seriam:

- Cliente
- Pedido
- Produto
- Pagamento
- Entrega

Exemplo:

```
public class Cliente {  
  
    private Long id;  
    private String nome;  
    private String email;  
  
    public Cliente(Long id, String nome, String email) {  
        this.id = id;  
        this.nome = nome;  
        this.email = email;  
    }  
  
    public Long getId() {  
        return id;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
  
    public String getEmail() {  
        return email;  
    }  
}
```

Observe que a classe representa apenas o estado e os comportamentos relacionados ao cliente.

Ela não possui qualquer conhecimento sobre telas ou requisições HTTP.

Componente View

A View é responsável pela apresentação das informações ao usuário.

Dependendo da tecnologia utilizada, uma View pode ser:

- página HTML;
- tela Desktop;
- aplicativo mobile;
- resposta JSON;

- relatório PDF.

No caso de uma API REST, normalmente a resposta JSON é considerada uma forma de View.

Exemplo:

```
{  
  "id": 1,  
  "nome": "Maria",  
  "email": "maria@email.com"  
}
```

A View não deve implementar regras de negócio.

Sua responsabilidade é apenas apresentar informações.

Componente Controller

O Controller atua como intermediário entre View e Model.

Suas principais responsabilidades são:

- receber requisições;
- validar entradas;
- chamar serviços de negócio;
- retornar respostas.

Exemplo conceitual:

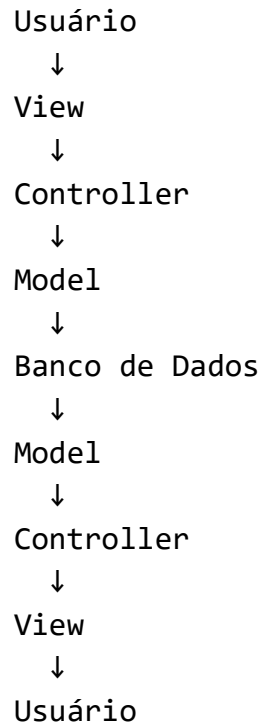
```
ClienteController  
recebe requisição  
  ↓  
ClienteService  
  ↓  
ClienteRepository  
  ↓  
Banco de Dados
```

O Controller não deve conter regras complexas de negócio.

Seu papel é coordenar o fluxo da aplicação.

Fluxo de Funcionamento do MVC

Em uma aplicação MVC típica, o fluxo ocorre da seguinte forma:

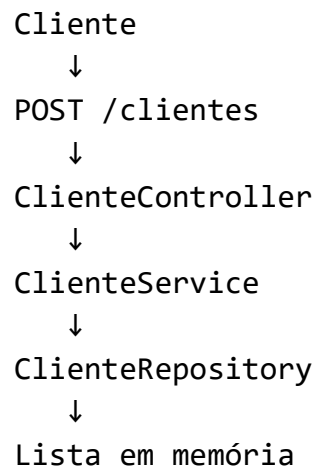


Esse fluxo garante separação de responsabilidades e facilita a manutenção.

MVC Aplicado ao Sistema de Entregas Inteligentes

Vamos implementar o caso de uso "Cadastrar Cliente".

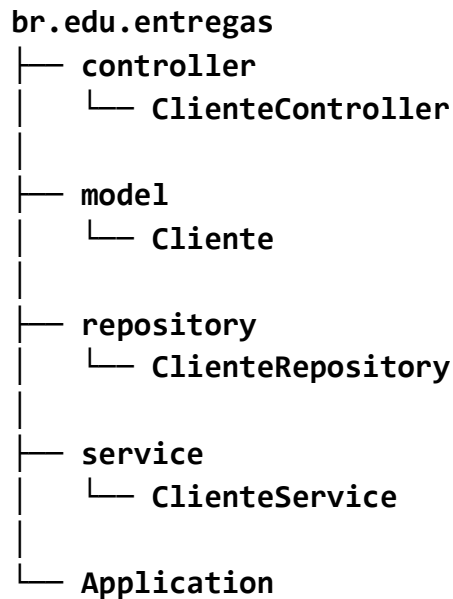
O fluxo será:



Esse exemplo servirá como base para uma API REST simples.

Estrutura do Projeto

Uma organização recomendada seria:



Essa estrutura favorece organização e crescimento do projeto.

Implementação do Model

```
package br.edu.entregas.model;

public class Cliente {

    private Long id;
    private String nome;
    private String email;

    public Cliente(Long id, String nome, String email) {
        this.id = id;
        this.nome = nome;
        this.email = email;
    }

    public Long getId() {
        return id;
    }
}
```

```

    }

    public String getNome() {
        return nome;
    }

    public String getEmail() {
        return email;
    }
}

```

Implementação do Repository

Inicialmente utilizaremos armazenamento em memória.

```

package br.edu.entregas.repository;

import java.util.ArrayList;
import java.util.List;
import br.edu.entregas.model.Cliente;

public class ClienteRepository {

    private List<Cliente> clientes = new ArrayList<>();

    public void salvar(Cliente cliente) {
        clientes.add(cliente);
    }

    public List<Cliente> listar() {
        return clientes;
    }
}

```

O Repository abstrai o acesso aos dados.

No futuro ele poderá ser substituído por um banco real sem impacto significativo nas demais camadas.

Implementação do Service

A camada de serviço concentra regras de negócio.

```
package br.edu.entregas.service;

import br.edu.entregas.model.Cliente;
import br.edu.entregas.repository.ClienteRepository;

import java.util.List;

public class ClienteService {

    private ClienteRepository repository =
        new ClienteRepository();

    public void cadastrar(
        Long id,
        String nome,
        String email) {

        if(nome == null || nome.isBlank()) {
            throw new RuntimeException(
                "Nome obrigatório");
        }

        Cliente cliente =
            new Cliente(id, nome, email);

        repository.salvar(cliente);
    }

    public List<Cliente> listar() {
        return repository.listar();
    }
}
```

Observe que a validação do nome foi implementada no Service e não no Controller.

Esse é um princípio importante do MVC.

Implementação do Controller

Exemplo simplificado:

```
package br.edu.entregas.controller;

import br.edu.entregas.service.ClienteService;

public class ClienteController {

    private ClienteService service =
        new ClienteService();

    public void cadastrarCliente(
        Long id,
        String nome,
        String email) {

        service.cadastrar(
            id,
            nome,
            email);
    }
}
```

Exemplo com Spring Boot

Em aplicações modernas geralmente utilizamos Spring MVC. Exemplo:

```
@RestController
@RequestMapping("/clientes")
public class ClienteController {

    private final ClienteService service;

    public ClienteController(
        ClienteService service) {

        this.service = service;
    }
}
```

```
@PostMapping
public void cadastrar(
    @RequestBody ClienteDTO dto) {

    service.cadastrar(
        dto.getId(),
        dto.getNome(),
        dto.getEmail());
}

@GetMapping
public List<Cliente> listar() {
    return service.listar();
}
}
```

Esse mesmo padrão poderá ser reutilizado nos módulos de:

- pedidos;
- entregas;
- pagamentos;
- produtos.

Benefícios do MVC

Entre os principais benefícios podemos destacar:

- Separação de Responsabilidades: Cada componente possui funções bem definidas.
- Facilidade de Manutenção: Alterações em uma camada têm impacto reduzido nas demais.
- Testabilidade: É possível testar Controllers, Services e Models de forma independente.
- Reutilização: A mesma lógica de negócio pode ser utilizada por diferentes interfaces.
- Organização: Projetos tornam-se mais compreensíveis para grandes equipes.

Limitações do MVC

Embora seja extremamente popular, MVC não é uma solução universal. Em sistemas muito pequenos ele pode introduzir complexidade desnecessária.

Também não resolve sozinho problemas relacionados a:

- escalabilidade;
- comunicação distribuída;
- microsserviços;
- integração corporativa.

Por esse motivo, nos próximos capítulos evoluiremos a arquitetura para modelos mais robustos.

Evolução do Estudo de Caso

Ao término deste capítulo, o Sistema de Entregas Inteligentes já possui:

- requisitos definidos;
- casos de uso modelados;
- classes identificadas;
- diagramas comportamentais;
- arquitetura documentada;
- protótipo inicial implementado utilizando MVC.

A próxima evolução será organizar essa implementação em uma estrutura arquitetural em camadas.

Resumo

O padrão MVC é uma das arquiteturas mais utilizadas no desenvolvimento de software. Ele divide a aplicação em Model, View e Controller, promovendo separação de responsabilidades, manutenção simplificada e maior organização do código. No estudo de caso da apostila, o MVC foi utilizado para construir o primeiro protótipo funcional do backend do Sistema de Entregas Inteligentes, servindo como base para as evoluções arquiteturais dos próximos capítulos.

Exercícios

1. Implemente a entidade Pedido seguindo o padrão MVC.

2. Crie um controller para gerenciamento de entregas.
3. Adicione validação de e-mail na camada de serviço.
4. Implemente operações de atualização e remoção de clientes.
5. Crie uma API REST para consulta de pedidos usando Spring Boot.
6. Refatore o protótipo desenvolvido para suportar persistência em banco de dados PostgreSQL.
7. Modele o caso de uso "Atualizar Status da Entrega" utilizando todas as camadas do MVC.

CAPÍTULO 7: EVOLUÇÃO PARA ARQUITETURA EM CAMADAS

Introdução

No capítulo anterior implementamos o Sistema de Entregas Inteligentes utilizando o padrão MVC. Essa abordagem permitiu organizar o software em Model, View e Controller, promovendo separação de responsabilidades e facilitando a manutenção do código.

Embora o MVC seja bastante eficiente para aplicações pequenas e médias, sistemas corporativos normalmente exigem uma organização adicional. À medida que novas funcionalidades são adicionadas, aumenta a necessidade de separar regras de negócio, persistência de dados, segurança, integração e infraestrutura.

Nesse contexto surge a Arquitetura em Camadas, um dos estilos arquiteturais mais utilizados em sistemas empresariais modernos. Ela organiza a aplicação em níveis distintos de responsabilidade, reduzindo acoplamento e aumentando a manutenibilidade do software.

O Que é Arquitetura em Camadas?

A arquitetura em camadas organiza o sistema em níveis hierárquicos, onde cada camada possui responsabilidades específicas. Normalmente uma camada interage apenas com as camadas imediatamente acima ou abaixo dela.

Esse modelo oferece diversos benefícios:

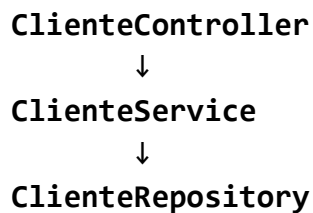
- Melhor organização do código;
- Separação de responsabilidades;

- Facilidade de manutenção;
- Maior reutilização;
- Facilidade para testes;
- Evolução simplificada da aplicação.

A principal ideia é que cada camada seja responsável por um conjunto específico de tarefas.

Problemas Encontrados no MVC Simples

Considere o projeto desenvolvido no capítulo anterior.



Essa estrutura funciona bem inicialmente.

Entretanto, quando começam a surgir:

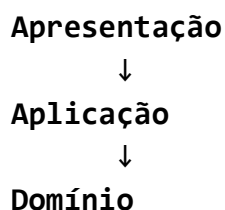
- validações complexas;
- controle de acesso;
- auditoria;
- integração com APIs externas;
- filas de mensagens;
- notificações;

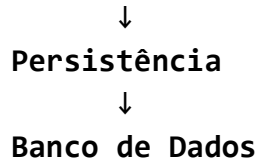
o sistema tende a crescer rapidamente e acumular responsabilidades.

A arquitetura em camadas surge justamente para controlar esse crescimento.

Estrutura Geral da Arquitetura em Camadas

Uma arquitetura corporativa típica costuma ser organizada da seguinte forma:





Dependendo da complexidade do projeto, também pode existir uma camada adicional de infraestrutura.

Camada de Apresentação

A camada de apresentação é responsável pela interação com usuários ou sistemas externos.

Dependendo da tecnologia adotada, ela pode ser representada por:

- páginas web;
- aplicações mobile;
- aplicações desktop;
- APIs REST;
- interfaces gráficas.

No Sistema de Entregas Inteligentes, os controllers Spring MVC pertencem à camada de apresentação.

Exemplo:

```
@RestController
@RequestMapping("/clientes")
public class ClienteController {

    @GetMapping
    public List<ClienteDTO> listar() {
        ...
    }
}
```

Essa camada não deve implementar regras de negócio.

Sua função é apenas receber e devolver informações.

Camada de Aplicação

A camada de aplicação coordena a execução dos casos de uso do sistema.

Ela funciona como uma orquestradora.

Suas responsabilidades incluem:

- iniciar operações;
- coordenar fluxos;
- controlar transações;
- chamar serviços de domínio;
- integrar componentes.

Exemplo:

```
public class PedidoApplicationService {  
  
    public Pedido criarPedido(  
        CriarPedidoCommand command) {  
  
        // coordena processo completo  
  
        return pedido;  
    }  
}
```

Observe que essa camada não contém necessariamente todas as regras de negócio.

Ela apenas coordena a execução.

Camada de Domínio

A camada de domínio contém o coração da aplicação.

Aqui ficam:

- entidades;
- objetos de valor;
- regras de negócio;
- validações complexas;
- políticas de negócio.

Exemplo:

```

public class Pedido {

    private List<ItemPedido> itens;

    public BigDecimal calcularTotal() {

        return itens.stream()
                    .map(ItemPedido::getSubtotal)
                    .reduce(BigDecimal.ZERO,
                            BigDecimal::add);
    }
}

```

Essa regra pertence ao negócio e não ao banco ou à interface.

Por isso ela deve permanecer no domínio.

Camada de Persistência

A camada de persistência é responsável pelo armazenamento e recuperação de dados.

Nesse nível encontramos:

- repositories;
- DAOs;
- consultas SQL;
- configurações ORM;
- integrações com bancos de dados.

Exemplo:

```

@Repository
public class ClienteRepository {

    public Cliente buscarPorId(Long id) {
        ...
    }

    public void salvar(Cliente cliente) {
        ...
    }
}

```

A camada de persistência não deve conhecer regras de negócio.

Sua única responsabilidade é manipular informações armazenadas.

Camada de Infraestrutura

Sistemas corporativos frequentemente possuem uma camada adicional denominada infraestrutura.

Ela concentra recursos técnicos utilizados por diversas partes da aplicação.

Exemplos:

- envio de e-mails;
- autenticação;
- autorização;
- logs;
- cache;
- filas de mensagens;
- integração com APIs externas;
- armazenamento em nuvem.

Exemplo:

```
public class EmailService {  
  
    public void enviarConfirmacao(  
        String email) {  
  
        ...  
    }  
}
```

O domínio não precisa saber como o e-mail é enviado.

Ele apenas solicita que essa operação seja realizada.

Arquitetura em Camadas Aplicada ao Estudo de Caso

Vamos reorganizar o Sistema de Entregas Inteligentes.

Estrutura proposta:

com.entregas

```
controller
├── ClienteController
├── PedidoController

application
├── ClienteApplicationService
├── PedidoApplicationService

domain
├── Cliente
├── Pedido
├── Produto
├── Entrega

repository
├── ClienteRepository
├── PedidoRepository

infrastructure
├── EmailService
├── LogService
```

Observe que a arquitetura ficou muito mais organizada do que no exemplo MVC inicial.

Fluxo de Criação de Pedido

Considere o caso de uso "Realizar Pedido".

O fluxo passa pelas camadas da seguinte forma:

```
Cliente
↓
Controller
↓
Application Service
↓
Domínio
↓
Repository
↓
```

Banco de Dados



Infraestrutura (Email)

Passo a passo:

- O cliente envia uma requisição.
- O Controller recebe os dados.
- O Application Service coordena a operação.
- O Domínio valida e cria o pedido.
- O Repository persiste os dados.
- A Infraestrutura envia uma confirmação.

Essa separação torna a aplicação mais robusta.

Benefícios da Arquitetura em Camadas

- Organização: Cada camada possui responsabilidades claramente definidas.
- Reutilização: A mesma lógica pode ser utilizada por diferentes interfaces.
- Manutenção: Mudanças ficam isoladas em regiões específicas do sistema.
- Testabilidade: As camadas podem ser testadas de forma independente.
- Escalabilidade Organizacional: Equipes diferentes podem trabalhar simultaneamente em partes distintas da aplicação.

Desvantagens da Arquitetura em Camadas

Nenhuma arquitetura é perfeita.

Algumas limitações incluem:

- Maior quantidade de código;
- Curva de aprendizado maior;
- Possível excesso de abstrações;
- Complexidade desnecessária em projetos pequenos.

Por isso é importante avaliar o contexto antes de aplicar qualquer padrão arquitetural.

Relação Entre MVC e Arquitetura em Camadas

É comum pensar que MVC e Arquitetura em Camadas são concorrentes.

Na prática isso não é verdade.

O MVC normalmente organiza a camada de apresentação, enquanto a Arquitetura em Camadas organiza todo o sistema.

Podemos visualizar da seguinte forma:

Apresentação
└─ MVC
Aplicação
Domínio
Persistência
Infraestrutura

Portanto, MVC e Camadas podem coexistir perfeitamente.

Preparação para Microsserviços

Até este ponto, o Sistema de Entregas Inteligentes continua sendo uma única aplicação.

Entretanto, à medida que o número de usuários cresce, surgem desafios relacionados a:

- escalabilidade;
- implantação;
- disponibilidade;
- independência entre equipes.

Esses problemas motivaram o surgimento da arquitetura de microsserviços.

No próximo capítulo veremos como evoluir nossa solução monolítica em camadas para uma arquitetura distribuída composta por múltiplos serviços independentes.

Resumo

A Arquitetura em Camadas organiza a aplicação em níveis de responsabilidade, promovendo separação de preocupações, manutenção facilitada e maior escalabilidade organizacional. Ela expande o modelo MVC estudado anteriormente e prepara o sistema para arquiteturas mais sofisticadas, como microsserviços. No Sistema de Entregas Inteligentes, a estrutura passou a incluir camadas de apresentação, aplicação, domínio, persistência e infraestrutura, criando uma base sólida para futuras evoluções.

Exercícios

1. Implemente uma camada de aplicação para o gerenciamento de pedidos.
2. Refatore o código MVC do capítulo anterior separando claramente aplicação, domínio e persistência.
3. Crie um serviço de envio de e-mails na camada de infraestrutura.
4. Implemente uma funcionalidade de auditoria utilizando uma camada de logs.
5. Desenhe o fluxo completo da operação "Realizar Pedido" indicando a participação de cada camada.
6. Analise quais componentes do Sistema de Entregas Inteligentes poderiam ser convertidos futuramente em microsserviços.
7. Proponha uma estrutura de pacotes para um sistema acadêmico utilizando Arquitetura em Camadas.

CAPÍTULO 8: EVOLUÇÃO PARA MICROSERVIÇOS

Introdução

Nos capítulos anteriores evoluímos gradualmente o Sistema de Entregas Inteligentes. Inicialmente identificamos os requisitos, modelamos os casos de uso, construímos modelos UML, documentamos a arquitetura por meio do modelo 4+1 e elaboramos o Documento de Arquitetura de Software. Posteriormente implementamos um protótipo baseado em MVC e reorganizamos o sistema utilizando Arquitetura em Camadas.

A arquitetura em camadas trouxe diversos benefícios, especialmente organização e separação de responsabilidades. Entretanto, à medida que sistemas crescem, surgem novos desafios:

- aumento do número de usuários;
- necessidade de escalabilidade;
- equipes maiores;
- implantação frequente;
- disponibilidade contínua;
- integração com sistemas externos.

Esses desafios levaram ao surgimento da arquitetura de microsserviços, um dos estilos arquiteturais mais adotados em sistemas distribuídos modernos.

O Que São Microserviços?

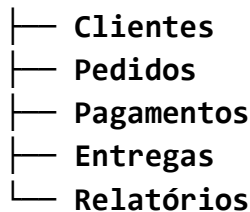
A arquitetura de microserviços organiza uma aplicação como um conjunto de serviços pequenos, independentes e especializados. Cada serviço é responsável por um contexto específico do negócio e pode ser desenvolvido, implantado e escalado de forma independente.

Ao contrário de um sistema monolítico, onde toda a aplicação está agrupada em um único projeto, os microserviços dividem as funcionalidades em múltiplos componentes autônomos.

Podemos pensar na seguinte analogia:

Sistema Monolítico

Sistema de Entregas



Tudo faz parte da mesma aplicação.

Arquitetura de Microserviços

Serviço de Clientes
Serviço de Pedidos
Serviço de Pagamentos
Serviço de Entregas
Serviço de Notificações

Cada elemento é um sistema independente.

Motivação para Utilizar Microserviços

Considere uma situação em que o Sistema de Entregas Inteligentes passa a atender milhões de usuários.

- Os pedidos aumentam rapidamente.
- Os pagamentos crescem.
- As consultas de rastreamento tornam-se constantes.

No modelo monolítico, para aumentar a capacidade de processamento seria necessário escalar toda a aplicação.

Isso significa consumir recursos adicionais até mesmo para funcionalidades que não precisam deles.

Com microsserviços é possível escalar apenas os componentes necessários.

Características dos Microsserviços

Algumas características são observadas na maioria das arquiteturas baseadas em microsserviços.

Independência

Cada serviço pode ser desenvolvido e implantado sem depender dos demais.

Exemplo:

- O serviço de entregas pode receber atualizações sem necessidade de recompilar o serviço de pagamentos.

Contexto de Negócio

Cada microsserviço representa uma área específica do domínio.

Exemplos:

- Clientes
- Pedidos
- Pagamentos
- Entregas
- Notificações

Essa abordagem reduz acoplamento e melhora a clareza da arquitetura.

Banco de Dados Próprio

Uma prática comum consiste em permitir que cada serviço possua sua própria persistência.

Pedidos

└ **Banco de Pedidos**

Pagamentos
└ **Banco de Pagamentos**
Entregas
└ **Banco de Entregas**

Isso reduz dependências diretas entre serviços.

Comunicação por APIs

Os serviços trocam informações utilizando protocolos de comunicação padronizados.

Exemplos:

REST
HTTP
Mensageria
Eventos
RabbitMQ
Kafka

De Monólito para Microsserviços

Nossa arquitetura atual está organizada em camadas.

Apresentação
Aplicação
Domínio
Persistência
Infraestrutura

Essa estrutura continuará existindo internamente.

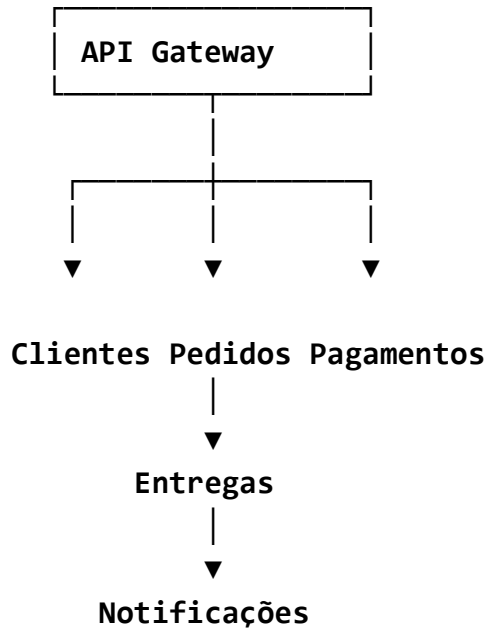
O que muda é que agora várias aplicações independentes serão criadas.

Sistema de Entregas
Clientes
Pedidos
Pagamentos
Entregas
Notificações

Tudo funciona dentro de uma única aplicação.

Arquitetura em Microsserviços

Após a evolução:



Cada componente pode evoluir independentemente.

Organização Interna dos Serviços

Um erro comum é imaginar que um microsserviço dispensa arquitetura interna.

Na prática, cada microsserviço geralmente continua utilizando:

- MVC;
- Arquitetura em Camadas;
- Padrões GoF;
- Boas práticas de projeto.

Por exemplo:

```
ServicoPedidos
  controller
  application
  domain
  repository
  Infrastructure
```

Ou seja, os conceitos estudados até o momento continuam válidos.

Serviço de Pedidos

Um possível microsserviço responsável pelos pedidos poderia disponibilizar:

```
POST /pedidos
GET /pedidos
GET /pedidos/{id}
DELETE /pedidos/{id}
```

Esse serviço teria total responsabilidade sobre o gerenciamento dos pedidos.

Nenhum outro serviço manipularia diretamente seus dados.

Serviço de Pagamentos

Responsável por:

- registrar pagamentos;
- validar transações;
- consultar status;
- integrar gateways financeiros.

Endpoints possíveis:

```
POST /pagamentos
GET /pagamentos/{id}
```

Serviço de Entregas

Responsável por:

- criar entregas;
- atualizar status;
- registrar localização;
- fornecer rastreamento.

Endpoints possíveis:

```
POST /entregas
GET /rastreamento/{codigo}
PUT /entregas/{id}
```

Serviço de Notificações

Responsável pelo envio de:

- e-mails;
- SMS;
- notificações push.

Uma alteração nesse serviço não impacta diretamente os demais sistemas.

Comunicação Entre Microsserviços

Existem duas formas principais de comunicação.

Comunicação Síncrona

O serviço solicita informações e aguarda resposta.

Exemplo:

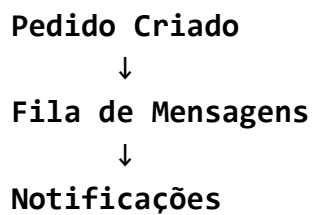


O serviço de pedidos envia uma requisição diretamente para pagamentos.

Comunicação Assíncrona

O serviço publica eventos sem esperar resposta imediata.

Exemplo:

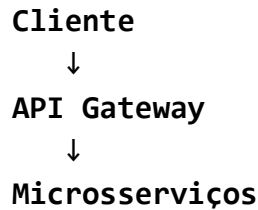


Essa abordagem reduz acoplamento entre componentes.

API Gateway

À medida que o número de serviços cresce, o cliente não deve acessá-los diretamente.

Por isso normalmente existe um API Gateway.



O Gateway centraliza:

- autenticação;
- roteamento;
- monitoramento;
- controle de acesso.

Vantagens dos Microserviços

- Escalabilidade: Cada serviço pode crescer de forma independente.
- Implantação Independente: Atualizações podem ocorrer sem interromper todo o sistema.
- Flexibilidade Tecnológica: Diferentes serviços podem utilizar tecnologias distintas.
- Resiliência: Falhas tendem a ficar isoladas em componentes específicos.
- Equipes Independentes: Diferentes equipes podem trabalhar simultaneamente em serviços distintos.

Desvantagens dos Microserviços

Apesar das vantagens, essa arquitetura também possui desafios.

- Complexidade Operacional: Administrar dezenas de serviços pode ser difícil.
- Monitoramento: É necessário monitorar múltiplos componentes simultaneamente.
- Testes Distribuídos: Os testes tornam-se mais complexos.
- Consistência de Dados: Manter dados consistentes entre serviços independentes requer planejamento arquitetural.

- Custos: A infraestrutura tende a ser mais cara.

Quando Utilizar Microserviços?

Microserviços costumam ser adequados quando:

- existem múltiplas equipes;
- o sistema possui elevada complexidade;
- diferentes partes apresentam necessidades distintas de escalabilidade;
- há grande volume de usuários;
- o software sofre mudanças frequentes.

Quando Não Utilizar Microserviços?

Nem todo sistema deve utilizar esse modelo.

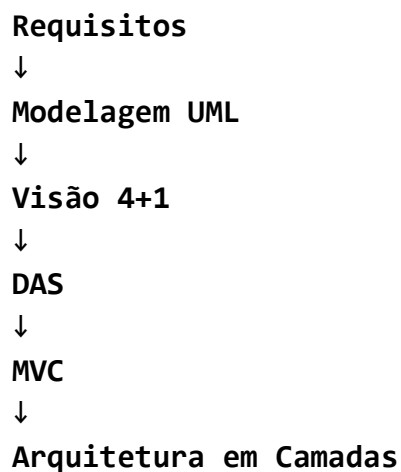
Projetos pequenos frequentemente obtêm melhores resultados com:

- MVC;
- Arquitetura em Camadas;
- Monólitos bem estruturados.

Aplicar microserviços prematuramente pode aumentar desnecessariamente a complexidade do projeto.

Evolução do Estudo de Caso

Com este capítulo, o Sistema de Entregas Inteligentes passou pela seguinte evolução arquitetural:





Microserviços

A aplicação agora encontra-se preparada para crescimento em larga escala e para atuação de múltiplas equipes de desenvolvimento.

Preparação para o Próximo Capítulo

Agora que a arquitetura foi distribuída em microserviços, surge uma nova preocupação:

- Como produzir código de qualidade dentro de cada serviço?
- Como reutilizar soluções para problemas recorrentes?
- Como evitar acoplamento excessivo e facilitar manutenção?

Essas questões serão respondidas no próximo capítulo, dedicado aos Padrões de Projeto GoF.

Resumo

A arquitetura de microserviços divide sistemas complexos em serviços independentes organizados em torno de contextos de negócio. Essa abordagem favorece escalabilidade, implantação independente e flexibilidade organizacional. No estudo de caso da apostila, o Sistema de Entregas Inteligentes evoluiu de uma aplicação monolítica em camadas para uma arquitetura distribuída composta por serviços de Clientes, Pedidos, Pagamentos, Entregas e Notificações.

Exercícios

1. Projete os microserviços necessários para um sistema de comércio eletrônico.
2. Defina os limites de negócio de cada serviço.
3. Proponha APIs REST para comunicação entre serviços.
4. Identifique quais serviços do Sistema de Entregas Inteligentes precisariam de maior escalabilidade.
5. Desenhe um fluxo de comunicação entre Pedidos, Pagamentos e Notificações.
6. Explique quando utilizar comunicação síncrona ou assíncrona.
7. Avalie se um sistema de biblioteca universitária realmente necessitaria de microserviços e justifique sua resposta.

CAPÍTULO 9: PADRÕES GoF E SUA APLICAÇÃO À ARQUITETURA DE SOFTWARE

Introdução

Ao longo dos capítulos anteriores, o Sistema de Entregas Inteligentes evoluiu gradualmente de um conjunto de requisitos para uma arquitetura distribuída baseada em microsserviços.

Entretanto, mesmo utilizando uma arquitetura bem definida, ainda surge uma questão importante:

Como projetar o código interno de cada componente de forma organizada, reutilizável e de fácil manutenção?

A resposta para esse problema está nos Padrões de Projeto, também conhecidos como Design Patterns.

Os padrões de projeto não são algoritmos, frameworks ou bibliotecas. Eles representam soluções reutilizáveis para problemas recorrentes encontrados durante o desenvolvimento de software. Diversos sistemas modernos utilizam esses padrões para reduzir acoplamento, aumentar reutilização e melhorar a qualidade arquitetural.

O conjunto mais conhecido de padrões foi documentado pelos autores Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides, conhecidos como Gang of Four (GoF).

O Que São Padrões de Projeto?

Um padrão de projeto pode ser definido como uma solução testada e validada para um problema recorrente de design de software.

Ao longo das últimas décadas, desenvolvedores identificaram que determinados problemas apareciam repetidamente:

Como criar objetos complexos?

- Como desacoplar componentes?
- Como permitir diferentes algoritmos?
- Como notificar partes interessadas do sistema?
- Como controlar acesso a recursos?

Os padrões GoF fornecem soluções para essas situações.

É importante compreender que um padrão não é um trecho de código pronto.

Ele representa uma estratégia de organização do software.

Benefícios dos Padrões GoF

A utilização adequada dos padrões oferece diversos benefícios.

Comunicação entre Desenvolvedores

Ao dizer que determinado componente utiliza o padrão Strategy, todos os membros da equipe entendem imediatamente a intenção do projeto.

Reutilização

Problemas semelhantes podem ser resolvidos de maneira consistente.

Manutenção

O código torna-se mais organizado e menos acoplado.

Evolução

Novas funcionalidades podem ser adicionadas sem grandes modificações em componentes existentes.

Alinhamento Arquitetural

Os padrões auxiliam na implementação prática das decisões arquiteturais documentadas no DAS.

Classificação dos Padrões GoF

Os 23 padrões do catálogo GoF são agrupados em três categorias.

- Padrões Criacionais: Tratam da criação de objetos.
- Padrões Estruturais: Tratam da composição de objetos e classes.
- Padrões Comportamentais: Tratam da interação entre objetos e distribuição de responsabilidades.

Padrões Criacionais

Singleton

O padrão Singleton garante que apenas uma instância de determinada classe exista durante toda a execução da aplicação.

Problema: Determinar que apenas um objeto de determinado tipo pode ser criado.

Aplicação Arquitetural: No Sistema de Entregas Inteligentes podemos utilizar Singleton para:

- Configurações da aplicação;
- Gerenciadores de cache;
- Serviços globais;
- Configuração de ambiente.

Exemplo

```
public class Configuracao {  
  
    private static Configuracao instancia;  
  
    private Configuracao() {}  
  
    public static Configuracao getInstancia() {  
  
        if(instancia == null) {  
            instancia = new Configuracao();  
        }  
  
        return instancia;  
    }  
}
```

Factory Method

O Factory Method encapsula a criação de objetos.

Problema: Evitar que o código cliente conheça detalhes de instanciação.

Aplicação Arquitetural: No Sistema de Entregas Inteligentes podemos criar diferentes tipos de notificações:

- E-mail;
- SMS;
- Push Notification.

```
public interface Notificacao {
    void enviar();
}
public class EmailNotificacao
    implements Notificacao {

    public void enviar() {
        System.out.println("E-mail");
    }
}
public class SmsNotificacao
    implements Notificacao {

    public void enviar() {
        System.out.println("SMS");
    }
}
```

Uma fábrica decide qual implementação criar.

Builder

Builder facilita a criação de objetos complexos.

Problema: Objetos possuem muitos parâmetros.

Aplicação Arquitetural: Um pedido pode possuir:

- cliente;
- endereço;
- produtos;
- desconto;
- frete;
- Observações.

Pedido pedido =

```
new PedidoBuilder()  
    .cliente(cliente)  
    .frete(frete)  
    .desconto(desconto)  
    .build();
```

Esse padrão melhora significativamente a legibilidade.

Prototype

Permite criar cópias de objetos existentes.

Aplicação Arquitetural: Pode ser utilizado para duplicação de:

- configurações;
- modelos de relatórios;
- templates de notificações.

Padrões Estruturais

Adapter

Permite integrar componentes incompatíveis.

Problema: Duas interfaces possuem formatos diferentes.

Exemplo Arquitetural: O Sistema de Entregas integra duas transportadoras.

Transportadora A:

```
enviarPedido()
```

Transportadora B:

```
processarEntrega()
```

O Adapter cria uma interface comum.

Benefício: Reduz impacto de integrações externas.

Facade

Fornece uma interface simplificada para subsistemas complexos.

Exemplo: Realização completa de uma compra.

Sem Facade:

Pagamento
Estoque
Entrega
Notificação

Com Facade:

```
lojaFacade.finalizarPedido();
```

Aplicação Arquitetural: Muito comum em:

- APIs;
- Camadas de aplicação;
- Serviços corporativos.

Proxy

Controla acesso a um objeto.

Aplicação Arquitetural: Pode ser utilizado para:

- cache;
- autenticação;
- autorização;
- lazy loading.

Exemplo:

ImagemProxy
somente carrega a imagem real quando necessário.

Decorator

Permite adicionar funcionalidades dinamicamente.

Exemplo

Frete base:
Frete
Frete com seguro:

Frete + Seguro

``

Frete com rastreamento premium:

Frete + Seguro + Rastreamento

Cada funcionalidade é adicionada sem alterar a implementação original.

Composite

Permite tratar objetos individuais e grupos de objetos da mesma forma.

Aplicação: Categorias de produtos.

Eletrônicos

└ Notebook

└ Tablet

Periféricos

└ Mouse

└ Teclado

Padrões Comportamentais

Strategy

Um dos padrões mais importantes para arquiteturas modernas.

Problema: Existem múltiplos algoritmos para o mesmo objetivo.

Exemplo: Cálculo de frete.

```
public interface CalculoFrete {  
    double calcular();  
}
```

Implementações:

FreteNormal

FreteExpresso

FreteInternacional

A escolha ocorre em tempo de execução.

Aplicação Arquitetural: Muito comum em:

- regras de negócio;
- cálculos;
- motores de precificação;
- meios de pagamento.

Observer

Define comunicação do tipo um-para-muitos.

Problema: Várias partes do sistema precisam ser notificadas quando um evento ocorre.

No Sistema de Entregas

Quando uma entrega muda de status:

Entrega Atualizada

↓

Email

SMS

Aplicativo

Todos recebem a atualização automaticamente.

Aplicação em Microsserviços: Esse padrão aparece fortemente em arquiteturas orientadas a eventos.

Command

Transforma operações em objetos.

Aplicação: Fila de processamento.

AtualizarEntregaCommand

EmitirCupomCommand

EnviarEmailCommand

Muito utilizado em:

- filas;
- mensageria;
- auditoria.

State

Permite alterar comportamento conforme o estado interno.

Exemplo: Entrega.

Estados possíveis:

- Criada
- Em Separação
- Em Transporte
- Entregue

Cada estado possui comportamentos específicos.

Esse padrão reduz grandes estruturas condicionais.

Template Method

Define a estrutura geral de um algoritmo.

Exemplo: Geração de relatórios.

- gerarRelatorio()
- Cabeçalho
- Corpo
- Rodapé

Cada formato implementa detalhes específicos.

- RelatorioPDF
- RelatorioHTML
- RelatorioCSV

Como os Padrões se Relacionam com a Arquitetura

Um erro comum é pensar que padrões GoF substituem uma arquitetura.

Na realidade eles operam em níveis diferentes.

- Arquitetura Define:
 - componentes;
 - serviços;
 - camadas;

comunicação.
Padrões Definem:
organização interna;
responsabilidades;
colaboração entre objetos.

Exemplo:

Microserviço de Pedidos. Internamente pode utilizar:

- Factory;
- Strategy;
- Observer;
- Facade.

Ou seja, os padrões ajudam a implementar a arquitetura definida anteriormente.

Aplicação dos Padrões ao Estudo de Caso

Ao final deste capítulo, o Sistema de Entregas Inteligentes utiliza:

- Singleton: Configuração global.
- Factory: Criação de notificações.
- Builder: Construção de pedidos.
- Facade: Finalização de compras.
- Strategy: Cálculo de fretes.
- Observer: Atualizações de rastreamento.
- Command: Processamento assíncrono.
- State: Gerenciamento do ciclo de vida das entregas.

Essa combinação resulta em uma solução mais flexível, reutilizável e aderente às boas práticas de engenharia de software.

Preparação para o Capítulo Final

Ao longo da disciplina construímos gradualmente um sistema completo:

Requisitos

↓

Casos de Uso

↓

Modelo de Classes
↓
Diagramas de Sequência
↓
Visão 4+1
↓
DAS
↓
MVC
↓
Arquitetura em Camadas
↓
Microserviços
↓
Padrões GoF

No próximo capítulo realizaremos uma consolidação de todos os conceitos por meio de um estudo de caso completo, articulando todos os artefatos produzidos ao longo da apostila.

Resumo

Os padrões GoF fornecem soluções reutilizáveis para problemas recorrentes de desenvolvimento. Quando utilizados adequadamente, reduzem acoplamento, aumentam reutilização e facilitam manutenção. Em arquiteturas modernas eles não substituem decisões arquiteturais, mas complementam a implementação, permitindo construir componentes mais robustos e flexíveis.

Exercícios

1. Explique a diferença entre Factory Method e Builder.
2. Implemente uma Strategy para cálculo de frete.
3. Modele um Observer para atualização de rastreamento.
4. Projete um Facade para finalização de pedidos.
5. Identifique oportunidades de utilização de Singleton no Sistema de Entregas Inteligentes.
6. Implemente um State para gerenciamento dos estados de entrega.
7. Analise quais padrões seriam mais úteis em um microserviço de pagamentos e justifique suas escolhas.

CAPÍTULO 10: ESTUDO DE CASO INTEGRADO: SISTEMA DE ENTREGAS INTELIGENTES

Introdução

Ao longo desta apostila acompanhamos a evolução de uma solução de software desde a identificação das necessidades do negócio até a definição de sua arquitetura e implementação.

Neste capítulo final, todos os conceitos estudados serão reunidos em um único estudo de caso integrado. O objetivo não é apresentar novos conceitos, mas demonstrar como requisitos, modelagem UML, arquitetura, documentação e implementação se relacionam em um projeto real.

O Sistema de Entregas Inteligentes será utilizado como exemplo central para consolidar os conhecimentos adquiridos durante a disciplina.

Contexto do Problema

Uma empresa de logística regional deseja desenvolver uma plataforma para gerenciamento e rastreamento de entregas.

Atualmente o processo é realizado manualmente, gerando diversos problemas:

- demora na atualização de informações;
- dificuldade de rastreamento;
- falhas de comunicação com clientes;
- falta de indicadores gerenciais;
- alto custo operacional.

O objetivo é desenvolver uma plataforma capaz de controlar todo o ciclo de vida das entregas.

Objetivos do Sistema

O sistema deverá permitir:

- cadastro de clientes;
- cadastro de produtos;
- registro de pedidos;
- processamento de pagamentos;

- rastreamento de entregas;
- envio de notificações;
- consulta de histórico;
- geração de relatórios.

Além disso, deverá suportar crescimento futuro da operação.

Levantamento de Requisitos

Requisitos Funcionais

- RF01 – Cadastrar clientes.
- RF02 – Atualizar cadastro de clientes.
- RF03 – Registrar pedidos.
- RF04 – Calcular valor do frete.
- RF05 – Processar pagamentos.
- RF06 – Criar entregas.
- RF07 – Atualizar status das entregas.
- RF08 – Consultar rastreamento.
- RF09 – Enviar notificações.
- RF10 – Emitir relatórios.

Requisitos Não Funcionais

- RNF01 – Disponibilidade mínima de 99%.
- RNF02 – Tempo de resposta inferior a três segundos.
- RNF03 – Comunicação segura utilizando HTTPS.
- RNF04 – Escalabilidade horizontal.
- RNF05 – Auditoria de operações críticas.
- RNF06 – Registro de logs.

Histórias de Usuário

- Cliente: Como cliente, eu quero acompanhar minha entrega para saber quando o pedido chegará.
- Entregador: Como entregador, eu quero atualizar o status da entrega para manter o cliente informado.
- Administrador: Como administrador, eu quero visualizar relatórios de desempenho para acompanhar a operação logística.

Casos de Uso Principais

Os principais casos de uso identificados foram:

Cliente

- |— Realizar Pedido
- |— Efetuar Pagamento
- |— Consultar Entrega
- |— Consultar Histórico

Administrador

- |— Gerenciar Clientes
- |— Gerenciar Produtos
- |— Emitir Relatórios
- |— Monitorar Operação

Entregador

- |— Atualizar Status
- |— Confirmar Entrega

Modelo de Classes

A partir dos requisitos foram identificadas as principais entidades.

- Cliente
 - id
 - nome
 - email
 - telefone
- Pedido
 - numero
 - data
 - valorTotal
 - status
 - calcularTotal()
 - cancelar()
- Produto
 - codigo
 - descricao
 - preco
- Pagamento

- valor
- dataPagamento
- status
- autorizar()
- cancelar()
- Entrega
 - codigoRastreamento
 - status
 - localizacaoAtual
 - atualizarStatus()
 - informarLocalizacao()

Modelo Simplificado

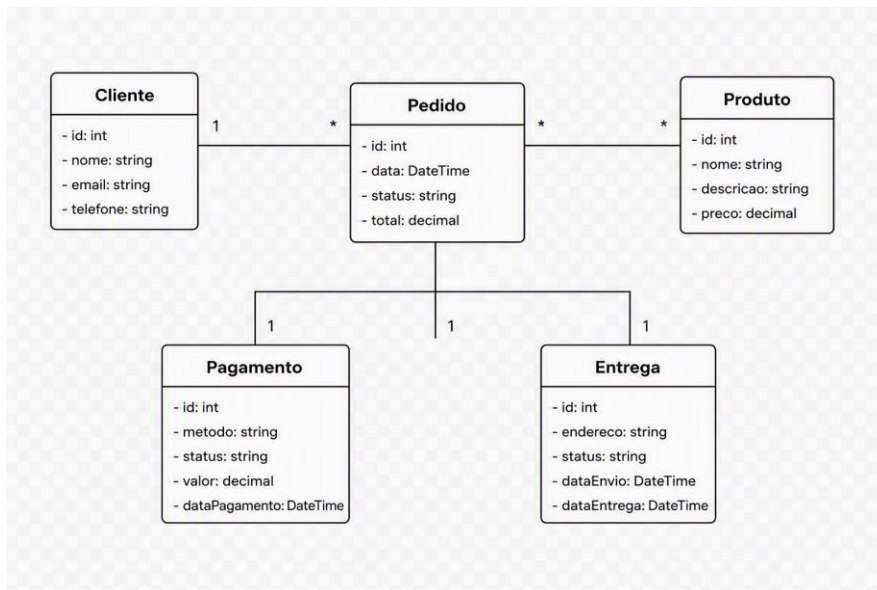
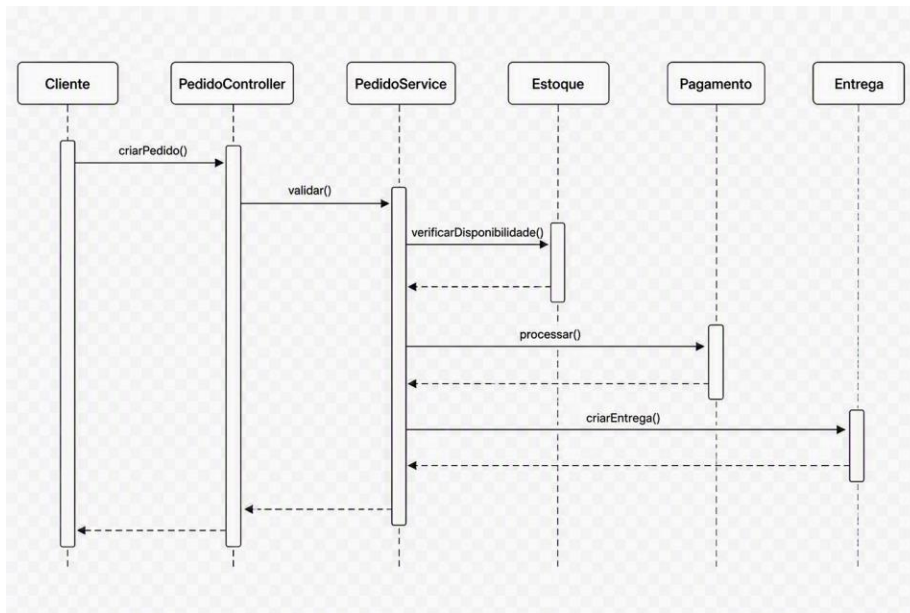


Diagrama de Sequência

Cenário: Realizar Pedido



Esse fluxo representa a interação entre os principais componentes do sistema.

Visão Arquitetural 4+1

Visão Lógica

Representa:

- Cliente
- Pedido
- Produto
- Pagamento
- Entrega

Visão de Processos

Representa:

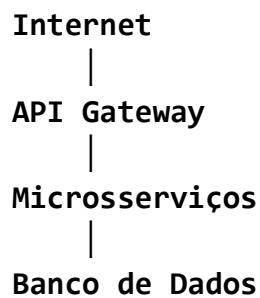
- Processamento de pedidos
- Comunicação com pagamentos
- Atualização de rastreamento
- Envio de notificações

Visão de Desenvolvimento

Organização dos módulos:

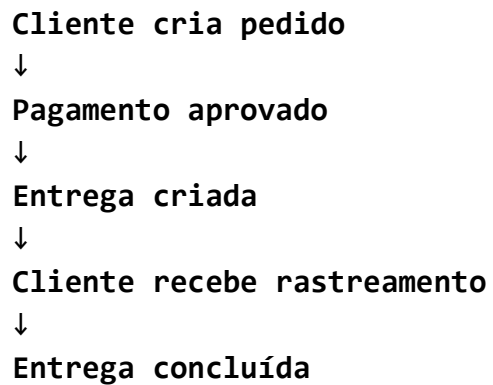
- clientes
- pedidos
- pagamentos
- entregas
- notificacoes
- Relatorios

Visão Física



Cenários

Cenário principal:



Documento de Arquitetura de Software

Objetivo: Definir a arquitetura do Sistema de Entregas Inteligentes.

Escopo: Aplicação web para gerenciamento e rastreamento de entregas.

Tecnologias

- Backend:
 - Java

- Spring Boot
- Banco:
 - PostgreSQL
- Comunicação:
 - REST
 - JSON
- Infraestrutura:
 - Docker
 - Kubernetes

Principais Decisões Arquiteturais

- Decisão 1: Utilizar Java e Spring Boot.
 - maturidade tecnológica;
 - ampla adoção corporativa;
 - produtividade.
- Decisão 2: Utilizar PostgreSQL.
 - robustez;
 - suporte transacional;
 - confiabilidade.
- Decisão 3: Evoluir para microsserviços.
 - crescimento previsto da empresa;
 - necessidade de escalabilidade.

Implementação Utilizando MVC

Model

Cliente
Pedido
Pagamento
Entrega
Produto

Controller

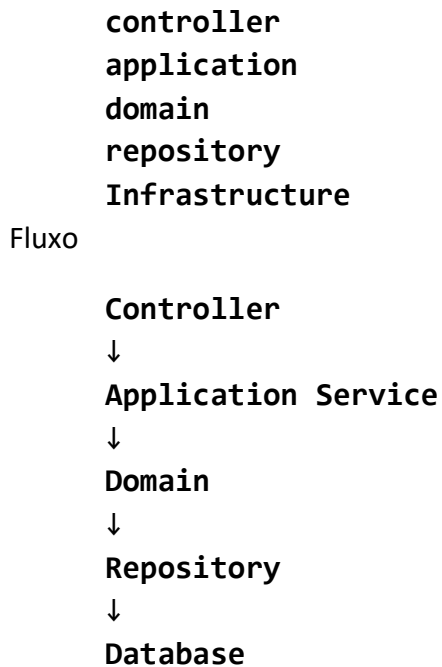
ClienteController
PedidoController
PagamentoController
EntregaController

View

Neste projeto a View será representada por respostas JSON consumidas por aplicações web e mobile.

Evolução para Arquitetura em Camadas

A aplicação foi reorganizada da seguinte forma:



Evolução para Microsserviços

Após o crescimento da empresa a solução foi dividida em cinco serviços.

ServicoClientes

ServicoPedidos

ServicoPagamentos

ServicoEntregas

ServicoNotificacoes

Cada serviço possui:

- banco próprio;
- API própria;

- implantação independente.

Aplicação dos Padrões GoF

- Singleton: Configurações globais da aplicação.
- Factory: Criação de notificações.
 - Email
 - SMS
 - Push
- Builder: Construção de pedidos complexos.
- Strategy: Cálculo de frete.
 - Frete Normal
 - Frete Expresso
 - Frete Internacional
- Observer: Atualização de rastreamento.

Entrega Atualizada

↓

Email

↓

Aplicativo

↓

Dashboard

- **State: Estados possíveis da entrega.**

Criada

↓

Separação

↓

Em Transporte

↓

Entregue

Avaliação Arquitetural

Ao final do projeto podemos observar como cada decisão influenciou a qualidade da solução.

Escalabilidade: Atendida por meio da decomposição em microserviços.

Manutenibilidade: Obtida através:

- MVC;
- Arquitetura em Camadas;
- Padrões GoF.

Flexibilidade: A utilização de Strategy, Factory e Observer reduziu acoplamento e facilitou evolução futura.

Disponibilidade: A arquitetura distribuída permite reduzir impactos causados por falhas localizadas.

Lições Aprendidas

Durante o desenvolvimento observamos que:

- requisitos bem definidos reduzem retrabalho;
- modelos UML facilitam comunicação;
- documentação arquitetural preserva conhecimento;
- arquiteturas devem evoluir conforme as necessidades do negócio;
- padrões de projeto complementam decisões arquiteturais;
- não existe arquitetura universalmente correta;
- toda decisão possui vantagens e desvantagens.

Conclusão da Apostila

A Arquitetura de Software é o processo de definir estruturas capazes de atender requisitos funcionais e não funcionais de uma organização.

Ao longo desta apostila foi demonstrada uma evolução arquitetural completa:

Levantamento de Requisitos

↓

Casos de Uso e Histórias de Usuário

↓

Modelo de Classes

↓

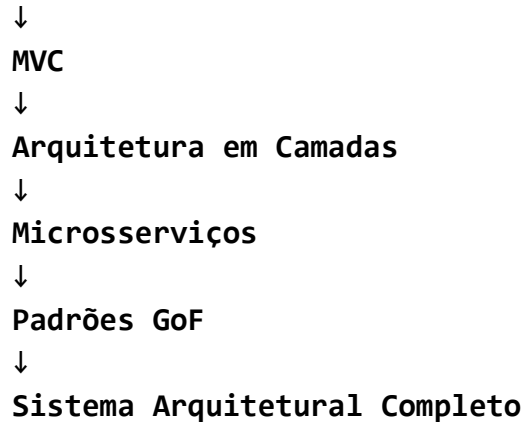
Diagramas de Sequência

↓

Visão 4+1

↓

Documento de Arquitetura



Mais importante do que memorizar diagramas ou padrões é compreender que a arquitetura deve sempre existir para resolver problemas reais de negócio.

Uma boa arquitetura não é necessariamente a mais complexa, mas sim a que atende adequadamente às necessidades do sistema, da equipe e da organização.

Exercício Integrador Final

1. Com base em todos os conteúdos da disciplina:
2. Elabore o DAS completo do Sistema de Entregas Inteligentes.
3. Desenhe os diagramas de classes e sequência.
4. Modele as cinco visões do modelo 4+1.
5. Implemente um protótipo MVC em Java.
6. Organize o sistema em camadas.
7. Proponha uma estratégia de migração para microserviços.
8. Identifique ao menos cinco padrões GoF aplicáveis ao projeto.
9. Justifique todas as decisões arquiteturais adotadas.